FRIEDRICH-SCHILLER-
UNIVERSITÄT
JENA

Datenbanksystemimplementierung

Prof. Dr. Viktor Leis

Professur für Datenbanken und Informationssysteme

DBMS Architecture

- so far, we mostly looked at the traditional database architecture
- developed between 1980 and 2010
- in the past decade many new systems have emerged that are very different from traditional ones
- many concepts are similar though
- change is mostly driven by hardware trends (e.g., multi-core CPUs, CPU caches, SIMD, NVMe SSDs)
- in this lecture, we will look at some historical and current developments

A Relational Model of Data for Large Shared Data Banks

E. F. Codd
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, inadequacies of these models

The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for noninferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

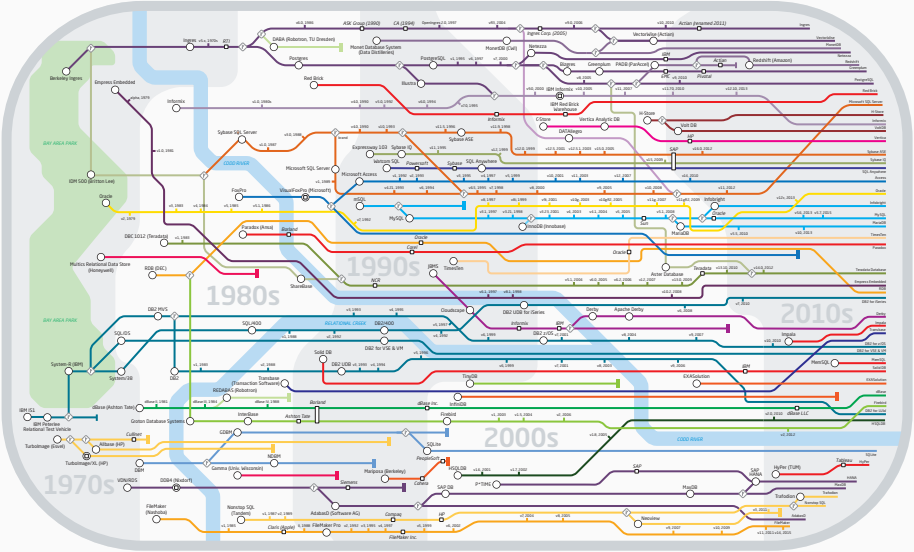
A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the "connection trap").

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted data systems, and also the relative merits (from a logical standpoint) of competing representations of data within a single system. Examples of this clearer perspective are

The “Dinosaurs”

- the earliest commercially available relational systems were Oracle, IBM System R, Ingres (end of 1970s)
- in 1990s and 2000s Oracle, MS SQL Server, IBM DB2 were the dominant players (in terms of mind- and market share)
- general-purpose systems: OLTP *and* OLAP
- behavior similar (all speak SQL), but different enough to make switching hard
- similar internal architecture
- stable, great functionality (ACID transactions, SQL)
- sometimes hard to use (database administrator needed), many tuning knobs to get better performance
- database systems research was kind of boring

Genealogy of Relational Database Management Systems



Felix Naumann, Jana Bauckmann, Claudia Exeler, Jan-Peer Rudolph, Fabian Tschirschnitz
Contact – Hasso Plattner Institut, University of Potsdam, felix.naumann@hpi.de
Design – Alexander Sarott Grafik-Design, Hamburg
Version – 2.0 (2015-12-31)
https://hpi.de/naumann/projects/rdbms-genealogy.html

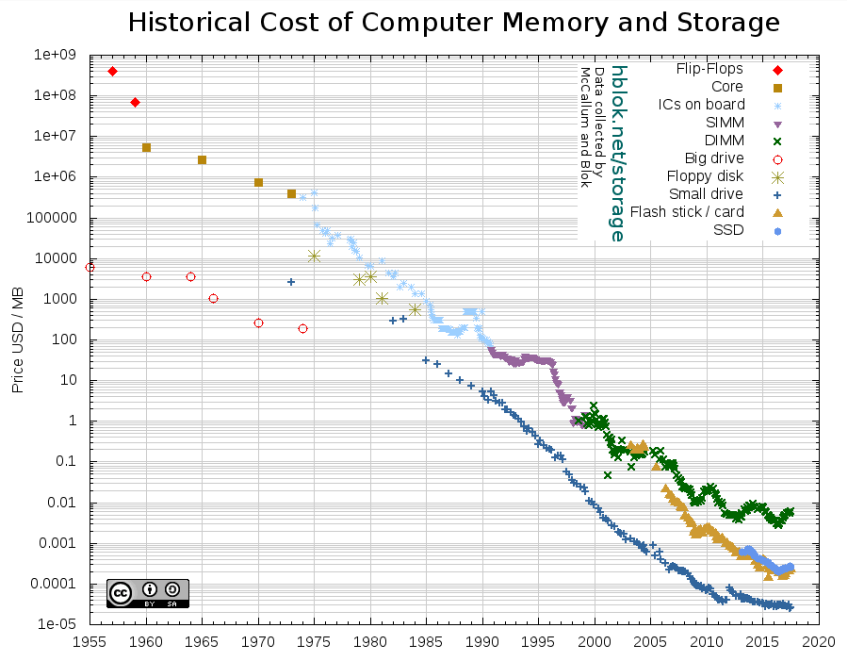
OLAP vs. OLTP

- Online Analytical Processing (OLAP)
  - mostly reads
  - long-running queries (one transaction per query)
  - many full table scans
  - batch inserts
  - example: compute revenue per month for last year
  - benchmark: TPC-H (also TPC-DS)
- Online Transactional Processing (OLTP)
  - many point writes and reads
  - short-running transactions (multiple statements per transaction)
  - heavy reliance on indexes
  - example: order processing (online shop)
  - benchmark: TPC-C (also TPC-E)

The Anatomy of a Dinosaur

feature	technique
transaction isolation	2 Phase Locking
synchronization	lock coupling
large data sets	buffer management
durability	ARIES-style logging
indexing	B+tree
storage	slotted pages (row-wise)
SQL	iterator model (interpreter)
parallelization	Exchange operators
query optimization	cost-based DP

- assumption: the only thing that matters for performance is minimizing disk I/O operations


Historical Cost of Computer Memory and Storage
Price USD / MB
1e+09
1e+08
1e+07
1e+06
100000
10000
1000
100
10
1
0.1
0.01
0.001
0.0001
1e-05
1955
1960
1965
1970
1975
1980
1985
1990
1995
2000
2005
2010
2015
2020
Flip-Flops
Core
ICs on board
SIMM
DIMM
Big drive
Floppy disk
Small drive
Flash stick / card
SSD
hblok.net/storage
Data collected by
McCallum and Blok

OLTP Through the Looking Glass, and What We Found There

Stavros Harizopoulos
HP Labs
Palo Alto, CA
stavros@hp.com

Daniel J. Abadi
Yale University
New Haven, CT
dna@cs.yale.edu

Samuel Madden Michael Stonebraker
Massachusetts Institute of Technology
Cambridge, MA
{madden, stonebraker}@csail.mit.edu

ABSTRACT

Online Transaction Processing (OLTP) databases include a suite of features — disk-resident B-trees and heap files, locking-based concurrency control, support for multi-threading — that were optimized for computer technology of the late 1970's. Advances in modern processors, memories, and networks mean that today's computers are vastly different from those of 30 years ago, such that many OLTP databases will now fit in main memory, and most OLTP transactions can be processed in milliseconds or less. Yet database architecture has changed little.

Based on this observation, we look at some interesting variants of conventional database systems that one might build that exploit recent hardware trends, and speculate on their performance through a detailed instruction-level breakdown of the major components involved in a transaction processing database system (Shore) running a subset of TPC-C. Rather than simply profiling

1. INTRODUCTION

Modern general purpose online transaction processing (OLTP) database systems include a standard suite of features: a collection of on-disk data structures for table storage, including heap files and B-trees, support for multiple concurrent queries via locking-based concurrency control, log-based recovery, and an efficient buffer manager. These features were developed to support transaction processing in the 1970's and 1980's, when an OLTP database was many times larger than the main memory, and when the computers that ran these databases cost hundreds of thousands to millions of dollars.

Today, the situation is quite different. First, modern processors are very fast, such that the computation time for many OLTP-style transactions is measured in microseconds. For a few thousand dollars, a system with gigabytes of main memory can be purchased. Furthermore, it is not uncommon for institutions to

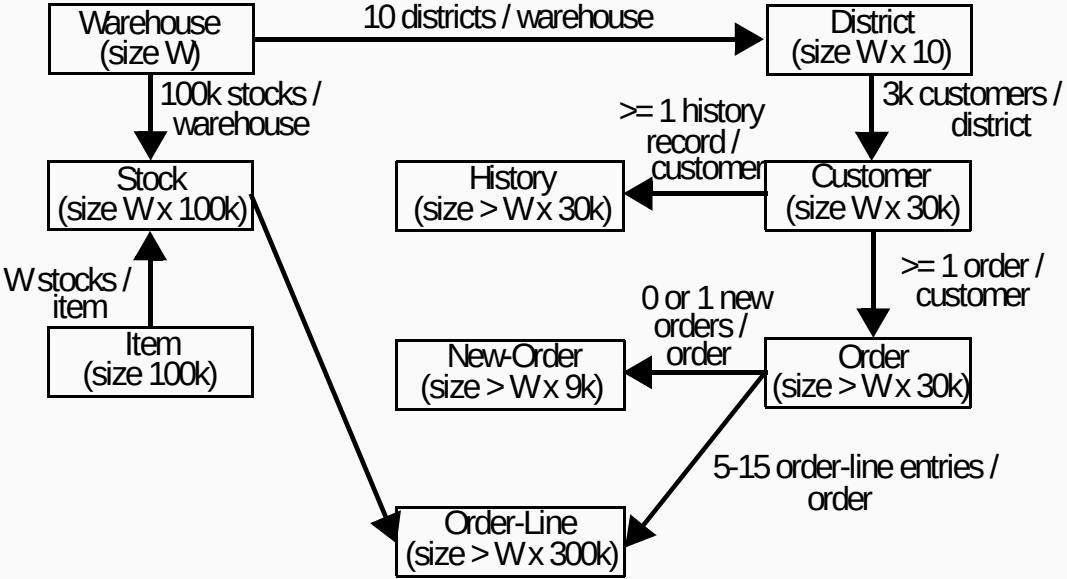
OLTP Through the Looking Glass [SIGMOD 2008]

- even a decade ago, the working set of many applications fit into main memory
- research question: Where does time go in OLTP?
- approach: disable/rip out components step by step (+ additional micro-optimizations)
- use Shore system
  - open source storage engine
  - developed at University of Wisconsin in early 1990s
  - architecturally similar to Dinosaurs
  - the assumption is that the results should be similar too

General Setup

- single-core Pentium 4, 3.2 GHz
- Linux
- measure instructions, cycles
- use TPC-C (standard OLTP benchmark)

TPC-C Schema


Warehouse
(size W)
10 districts / warehouse
District
(size W x 10)
100k stocks /
warehouse
3k customers /
district
>= 1 history
record /
customer
Stock
(size W x 100k)
History
(size > W x 30k)
Customer
(size W x 30k)
W stocks /
item
>= 1 order /
customer
0 or 1 new
orders /
order
Item
(size 100k)
New-Order
(size > W x 9k)
Order
(size > W x 30k)
5-15 order-line entries /
order
Order-Line
(size > W x 300k)

TPC-C Transactions

New Order

begin
for loop(10)
.....Btree lookup(I), pin
Btree lookup(D), pin
Btree lookup (W), pin
Btree lookup (C), pin
update rec (D)
for loop (10)
.....Btree lookup(S), pin
.....update rec (S)
.....create rec (O-L)
.....insert Btree (O-L)
create rec (O)
insert Btree (O)
create rec (N-O)
insert Btree (N-O)
insert Btree 2ndary(N-O)
commit

Payment

begin
Btree lookup(D), pin
Btree lookup (W), pin
Btree lookup (C), pin
update rec (C)
update rec (D)
update rec (W)
create rec (H)
commit

Instruction Breakdown

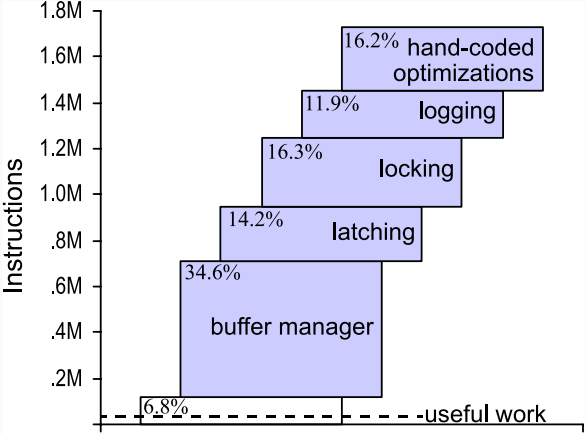


Figure 1. Breakdown of instruction count for various DBMS components for the New Order transaction from TPC-C. The top of the bar-graph is the original Shore performance with a main memory resident database and no thread contention. The bottom dashed line is the useful work, measured by executing the transaction on a no-overhead kernel.

Transactions Per Second

- out-of-the-box Shore: 640
- disable log flushing: 1,700
- disable components: 12,700
- standalone C implementation: 46,500

OLTP Through the Looking Glass: Conclusions

- traditional database implementation and architecture can be extremely inefficient
- 10× to 100× performance gains are achievable
- all components are slow (“no high pole in the tent”)

The End of an Architectural Era
(It's Time for a Complete Rewrite)

Michael Stonebraker
Samuel Madden
Daniel J. Abadi
Stavros Harizopoulos
MIT CSAIL
{stonebraker, madden, dna, stavros}@csail.mit.edu

Nabil Hachem
AvantGarde Consulting, LLC
nhachem@agdba.com

Pat Helland
Microsoft Corporation
phelland@microsoft.com

ABSTRACT

In previous papers [SC05, SBC+07], some of us predicted the end of "one size fits all" as a commercial relational DBMS paradigm. These papers presented reasons and experimental evidence that showed that the major RDBMS vendors can be outperformed by 1-2 orders of magnitude by specialized engines in the data warehouse, stream processing, text, and scientific database markets.

Assuming that specialized engines dominate these markets over time, the current relational DBMS code lines will be left with the business data processing (OLTP) market and hybrid markets where more than one kind of capability is required. In this paper we show that current RDBMSs can be beaten by nearly two

All three systems were architected more than 25 years ago, when hardware characteristics were much different than today. Processors are thousands of times faster and memories are thousands of times larger. Disk volumes have increased enormously, making it possible to keep essentially everything, if one chooses to. However, the bandwidth between disk and main memory has increased much more slowly. One would expect this relentless pace of technology to have changed the architecture of database systems dramatically over the last quarter of a century, but surprisingly the architecture of most DBMSs is essentially identical to that of System R.

Moreover, at the time relational DBMSs were conceived, there was only a single DBMS market, business data processing. In the last 25 years, a number of other markets have evolved, including

The End of an Architectural Era (It's Time for a Complete Rewrite) [ICDE 2007]

- Stonebraker's lesson:
  - existing code bases are hopeless, rewrite needed
  - specialized, simplified systems for OLTP, OLAP, text, etc.
  - let's start lots of startups building specialized systems (OLTP: H-Store, OLAP: C-Store/Vertica, Data Integration/Cleaning: Tamr)
- German lesson:
  - general-purpose relational systems are kind of nice
  - DRAM is cheap (1 TB RAM for less than 50K EUR), let's go in-memory only
  - SAP HANA, TUM HyPer

Modern Systems

- Column Stores for OLAP
  - Actian Vector (“Vectorwise”)
  - Microsoft Apollo (part of SQL Server)
  - IBM BLU
- OLTP (in-memory)
  - Microsoft Hekaton (part of SQL Server)
  - VoltDB
- OLTP and OLAP (in-memory)
  - SAP HANA
  - TUM HyPer

DBMS Evolution

feature	old	new
transaction isolation	2 Phase Locking	MVCC
synchronization	lock coupling	optimistic lock coupling
large data sets	buffer management	pointer swizzling
durability	ARIES-style logging	scalable logging
indexing	B+tree	B+tree/trie
storage	slotted pages (row-wise)	column stores
SQL	iterator model (interpreter)	compilation or vectorization
parallelization	Exchange operators	morsel-driven parallelism
query optimization	cost-based DP	cost-based DP

Conclusions

- fast changes in hardware drive evolution of database systems
- many new techniques
- concepts stay similar, but need to be rethought
- big trend: cloud
- database systems can never be fast enough